

Online

Computing

DBMS

Business

CSE3001 — Database
Management Systems

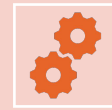
Objectives



Advantages of using a DBMS rather than a file system.



Designing an Entity-Relationship model for a real life application.



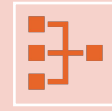
Mapping a database schema from ER model.



Evaluating relational schemas for design qualities



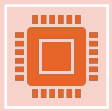
Optimize a query.



Basic concepts on transaction processing, concurrency control and recovery.



Fundamental view on unstructured data and its management.



Storage of databases and techniques to access them using various algorithms

Expected Outcomes

- Explain the role of a database management system in an organization.
- Describe the structure and operation of the relational data model.
- Construct database queries using Structured Query Language (SQL).
- Design and implement a database project depending on the business requirements, considering various design issues.
- Implement the concept of a database transaction and related database facilities, including concurrency control, backup and recovery.

Syllabus

Unit – 1

- **Introduction to Database system:** purpose of database system, view of data, Independence, relational databases, -Database Language – Database System architecture- levels, Mappings, Database, users and DBA Data Models: Importance, Basic building blocks, Degrees of data abstraction Database design and ER Model: Overview, ER-Model, Constraints, ER-Diagrams, ERD Issues, weak entity sets.

Unit – 2

- **Relational Models:** Structure of relational databases, Domains, Relations, Relational algebra – fundamental operators and syntax, selection, Projection, relational algebra queries, tuple relational calculus, set operations, renaming, Joins, Division, syntax. Operators, grouping and ungrouping, relational comparison. Codd's rules, Relational Schemas, Introduction to UML
Relational database model: Logical view of data, keys, integrity rules. **Normalization:** 1NF, 2NF, 3NF, BCNF, Multi valued dependencies and Fourth Normal Form.

Syllabus

Unit – 3

- Relational Database design: features, atomic domain. SQL: data definition, aggregate function, Null Values, nested sub queries, Joined relations. Triggers. Data Manipulation Language (DML) and Transaction Control Language (TCL) , Basic SELECT statements , Table Joins, Restricting and Sorting Data - Order By / Group By. Retrieving Data Using the SQL SELECT Statement, Single-Row Functions, Conversion Functions and Conditional Expressions, Reporting Aggregated Data Using the Group Functions, Displaying Data from Multiple Tables. Joins, Set Operators, DML Statements, Data Definition Language - Data Dictionary Views - Creating Sequences, Synonyms, and Indexes, Creating Views - Introduction to views, data independence, security, updates on views, comparison between tables and views.

Syllabus

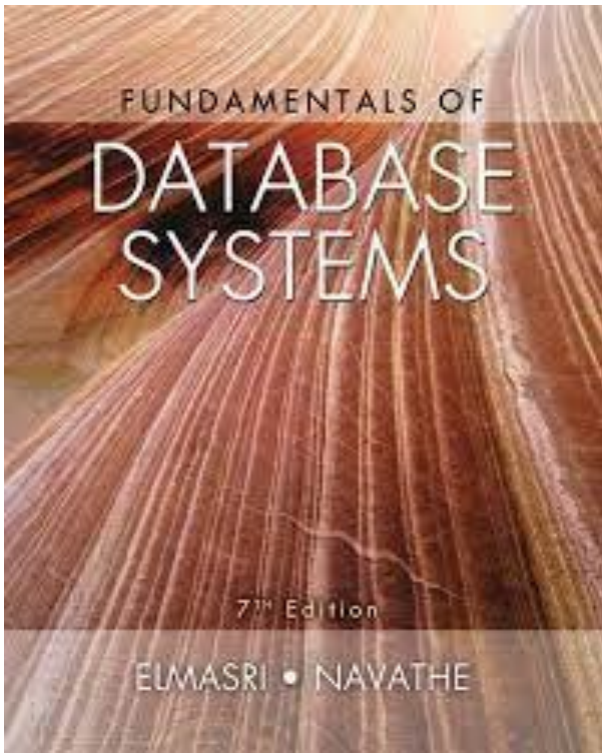
Unit – 4

- **PL/SQL:** Declaring PL/SQL Variables, Writing Executable Statements, Using SQL Statements Within a PL/SQL Block, Control Structures, Composite Data Types, Using Explicit Cursors, Handling Exceptions, Introducing Stored Procedures and Functions. **Data Storage:** Overview of Physical Storage Media - Magnetic disk Flash storage -RAID-File and Record Organization-Indexing and Hashing, Ordered Indices - B+Tree Index File-Static Hashing -Dynamic Hashing- Query Processing: Overview-measures of Query Cost. Query Optimization Techniques- Cost based Optimization-Heuristic Optimization

Unit – 5

- **Transaction Management:** Transaction concepts-properties of transactions-Transaction Atomicity and Durability-Transaction Isolation-Implementation of Isolation Levels-serializability of transactions, testing for serializability. **Concurrency Control:** Lock based protocols--Deadlock handling-Multiple Granularity-Time stamp based protocols -System recovery, Two- Phase Commit protocol. **Recovery system:** Failure classification -- Recovery and atomicity - Recovery Algorithms.

Textbooks & Reference Books



- Textbooks
 - R. Elmasri & S. B. Navathe, Fundamentals of Database Systems, Addison Wesley, 7th Edition, 2015
 - Raghu Ramakrishnan, Database Management Systems, McGraw Hill, 4th edition, 2015
- Reference Books
 - A. Silberschatz, H. F. Korth & S. Sudershan, Database System Concepts, McGraw Hill, 6th Edition 2010
 - Thomas Connolly, Carolyn Begg," Database Systems : A Practical Approach to Design, Implementation and Management", 6th Edition, 2010

APR
2024

OVERVIEW OF SOCIAL MEDIA USE

HEADLINES FOR SOCIAL MEDIA ADOPTION AND USE (NOTE: USER IDENTITIES MAY NOT REPRESENT UNIQUE INDIVIDUALS)



NUMBER OF SOCIAL
MEDIA USER IDENTITIES



5.07
BILLION

QUARTER-ON-QUARTER CHANGE
IN SOCIAL MEDIA USER IDENTITIES



+0.7%
+37 MILLION

YEAR-ON-YEAR CHANGE IN
SOCIAL MEDIA USER IDENTITIES



+5.4%
+259 MILLION

AVERAGE DAILY TIME SPENT
USING SOCIAL MEDIA



2H 20M
YOY: -2.7% (-4 MINS)

AVERAGE NUMBER OF SOCIAL
PLATFORMS USED EACH MONTH



6.7
YOY: +1.5% (+0.1)

SOCIAL MEDIA
USER IDENTITIES vs.
TOTAL POPULATION



62.6%

SOCIAL MEDIA USER
IDENTITIES AGED 18+ vs.
POPULATION AGED 18+



84.3%

SOCIAL MEDIA USER
IDENTITIES vs. INDIVIDUALS
USING THE INTERNET



93.3%

FEMALE SOCIAL MEDIA USER
IDENTITIES vs. TOTAL SOCIAL
MEDIA USER IDENTITIES



46.6%

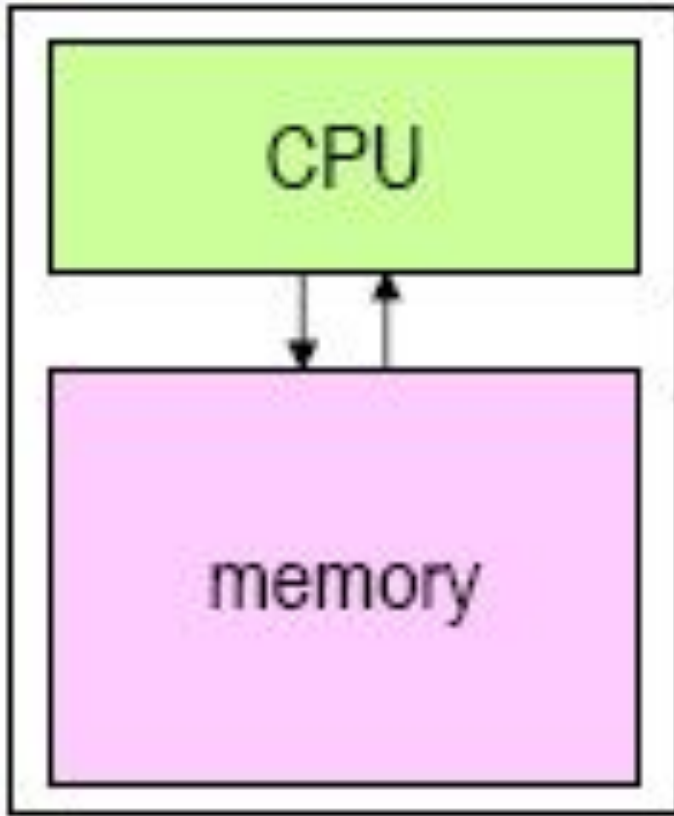
MALE SOCIAL MEDIA USER
IDENTITIES vs. TOTAL SOCIAL
MEDIA USER IDENTITIES



53.4%

SOURCES: KEPIOS ANALYSIS; COMPANY ADVERTISING RESOURCES; CNNIC; BETA RESEARCH CENTER; OCDH; U.N.; GWI (Q4 2023). **NOTE:** AVERAGE NUMBER OF PLATFORMS INCLUDES DATA FOR YOUTUBE. **ADVISORY:** SOCIAL MEDIA USER IDENTITIES MAY NOT REPRESENT UNIQUE INDIVIDUALS. COMPARISONS WITH POPULATION AND INTERNET USERS MAY EXCEED 100% DUE TO DUPLICATE AND FAKE ACCOUNTS, USER AGE MISSTATEMENTS, DIFFERENT REPORTING PERIODS, AND DIFFERENCES BETWEEN CENSUS COUNTS AND RESIDENT POPULATIONS. **COMPARABILITY:** SOURCE AND METHODOLOGY CHANGES; BASE REVISIONS. SEE [NOTES ON DATA](#).

Storing Data: Memory

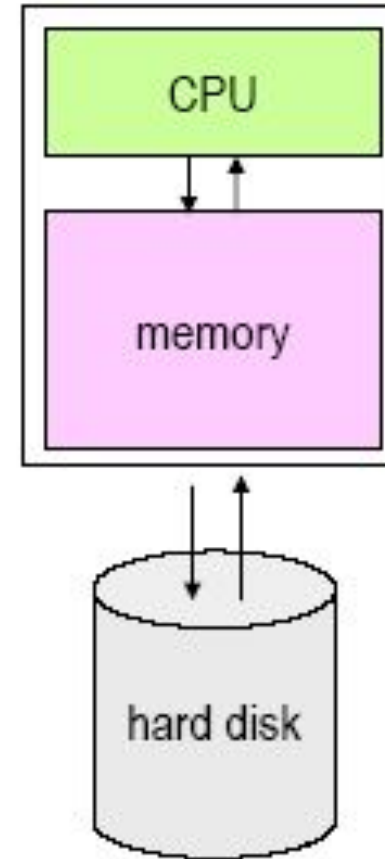


- Memory is used to store programs and other data that are currently in use.
 - LOAD, STOR operations
- Advantage of memory: short *access times*
 - Read/write times in nanoseconds (10^{-9} sec)
- Disadvantages of memory:
 - relatively expensive (\$/byte)
 - “volatile”

Storing Data: Secondary Storage

Secondary storage is used to store data for later use (examples: disks, CD, DVD).

- Data is written from memory to disk.
- When needed, data is read back into memory.



Secondary Storage



Advantages of secondary storage:

- relatively inexpensive (\$/byte)
- not “volatile”

Disadvantage of secondary storage: long access times

- Read times in milliseconds (10^{-3} sec).
- in 10 ms, a modern CPU can perform millions of operations!

Thus, it's important to minimize the number of times that the disk is accessed.

Example: Instagram Profile

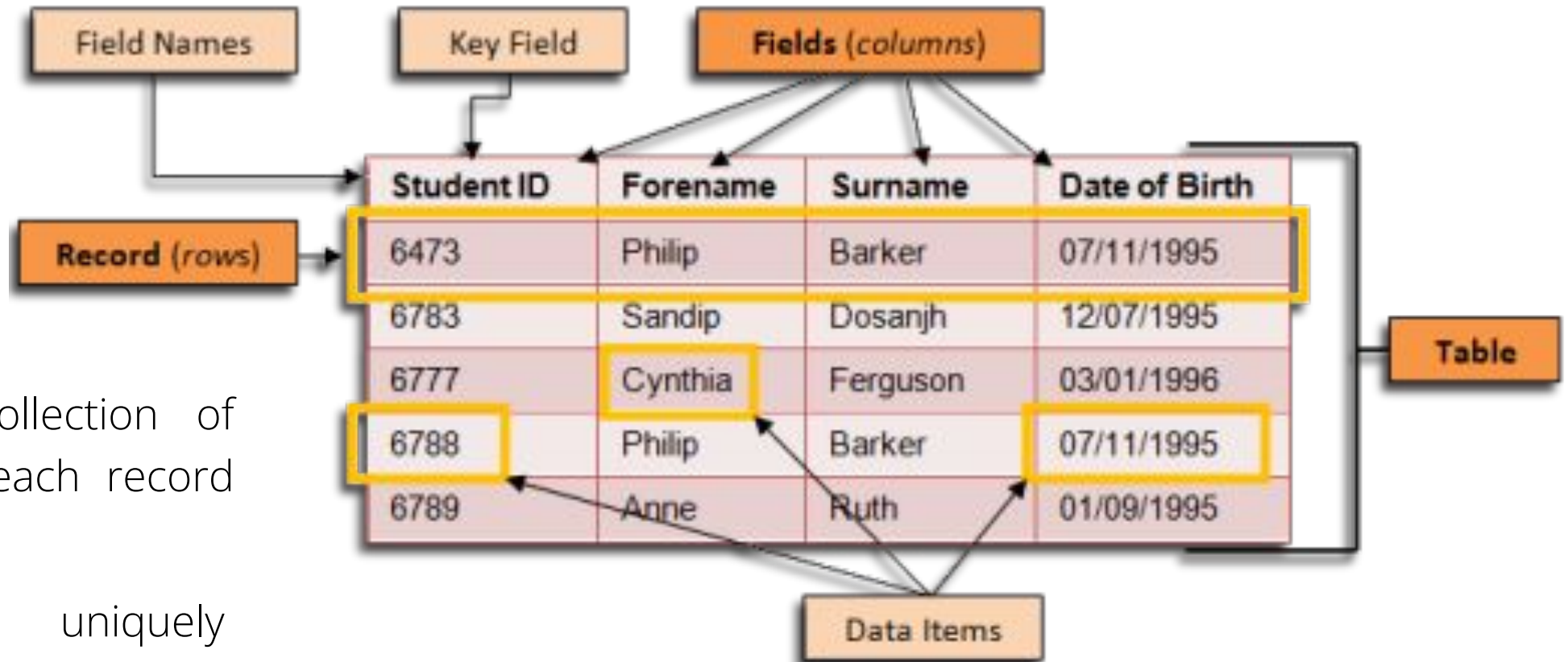
What info is on a for profile page?

First Name	Activities
Last Name	Favorite Books
Email	Favorite Movies
Password	Favorite Music
Birthday	Favorite Quotes
About me	Favorite TV shows

No matter which page you view, it has these same elements...

These data are stored in a database **table**.

What is a Table?



- Table is a collection of records, and each record contains fields.
- The Key Field uniquely identifies one record within a table.

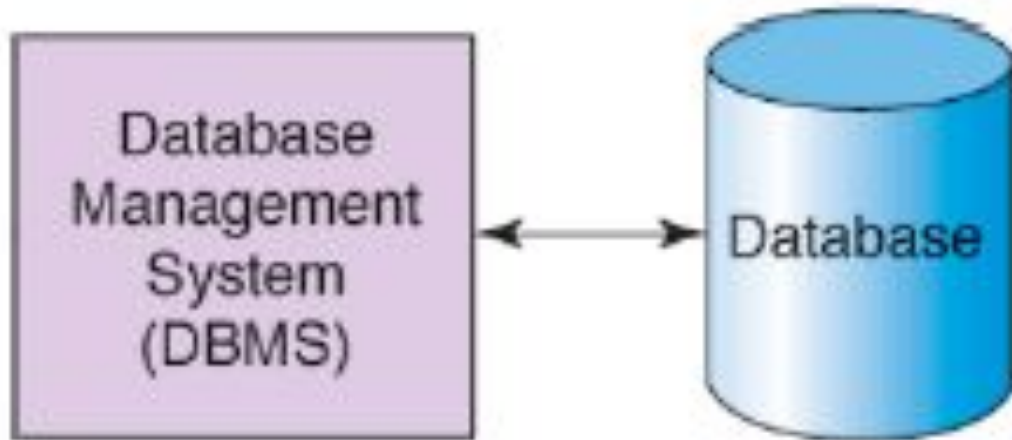
What is a Database?

A database is:

- a collection of data stored in a way to enable quick access
- organized into related sub-collections called tables.



Database Management Systems



- A database is a collection of data (not software).
- A database management system (**DBMS**) is the software which manages a database.
- Functions of a **DBMS**:
 - Efficient storage and access
 - Providing a logical view of data (tables, records)
 - Query processing
 - Transaction management

Function: Efficient Storage and Access

Indexing enables efficiently locating a record based on some unique attribute, called a key.

- Example: looking up stocks by their `symbol`.
- Example: update the record for `symbol= 'FB'`

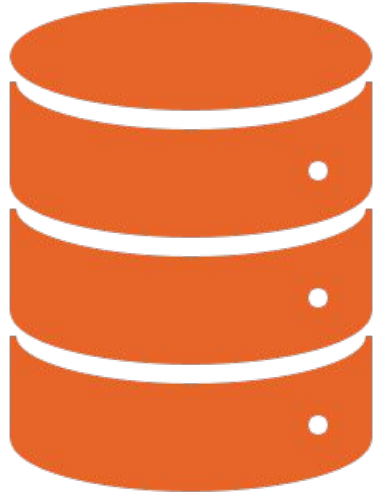
A hash index can make these operations $O(n)$

A b-tree index can make these operations $O(\log n)$

Function: Logical View

Abstraction – Representing the data as tables and rows, so programmers (usually) don't need to think about how the data is physically stored on disk (e.g., in which files?)

Function: Query Processing



A query language is used to access and modify the data.

SQL (Structured Query Language) is the standard for relational databases.

Many different database vendors support SQL:

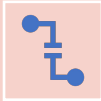
- Oracle, Sybase, IBM DB2, MS SQL Server, MS Access
- PostgreSQL, MySQL, SQLite (free/open-source)

Function: Transaction Management



Atomicity

All transaction(s) must be completed, or none of it



Consistency (Correctness)

Don't allow the database to enter a corrupted state



Isolation

Concurrently executed transactions must have the same effects as sequentially executed transactions (since databases usually have multiple users)



Durability

After a transaction completes, it should persist even if the power fails, etc.

What is a Database System?

- Database:
 - A very large collection of related data
- Models a real world enterprise:
 - Entities (e.g., teams, games / students, courses)
 - Relationships (e.g., The Celtics are playing in the Final!)
 - Even active components (e.g. "business logic")
- DBMS: A software package/system that can be used to store, manage and retrieve data from databases
- Database System: DBMS+data (+ applications)

Why Study Databases?

- Shift from computation to information
- Always true for corporate computing
- More and more true in the scientific world and of course, Websites
- New trends: social media generate ever increasing amount of data
- DBMS encompasses much of CS in a practical discipline
 - OS, languages, theory, AI, logic

Advanced Topics

- Data Analytics is an important domain
 - Given large amount of data in the database, how can you use them to help you take business decisions?
- Cluster based data analytics:
 - Map-Reduce, shared nothing DBs
- Main memory databases (NVRAM)

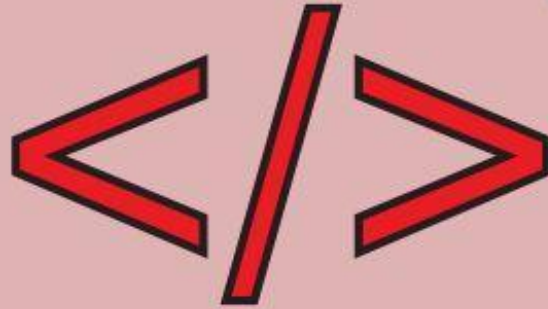
Outline

- Application-oriented
 - How to develop database applications: User + DBA
 - For learning, we will do things the hard way (i.e., using lightweight frameworks). In the real world, I recommend a featureful framework like Django, but doing things the hard way will show the value of more featureful frameworks.
- System-oriented
 - Learn the internals of a relational DBMS

UNIT 1

Introduction to Database system: purpose of database system, T1:View of data, Data Independence(7), relational databases-A1:Database Languages(4)-T2:Database System architecture- levels, Mappings, Database, users and DBA Data Models: Importance, Basic building blocks, T3:Degrees of data abstraction(3). Database design(3) and A1:ER Model: Overview, ER-Model, Constraints, ER Diagrams, ERD Issues, weak entity sets. (understand from the slide)

PURPOSE OF DATABASE SYSTEM



-
- When we talk about a database system, we're diving into the world of structured data storage and retrieval – an essential aspect for modern technology.
 - The primary purpose of a database system is to efficiently store, manage, and retrieve data.
 - Firstly, database systems offer a structured environment where data is organized in a way that makes it easily accessible.
 - Additionally, database systems ensure data integrity and security. They have mechanisms to prevent data corruption and unauthorized access, which is crucial, especially when dealing with sensitive information.
 - Another key aspect is data management efficiency. Database systems allow multiple users to access and manipulate data simultaneously without compromising its integrity. This is achieved through transaction management, which ensures that all database transactions are processed reliably and in a logically coherent manner.

Basic Definitions:

What is data, database, DBMS??

- Data: Known facts that can be recorded and have an implicit meaning; raw
- Database: a highly organized, interrelated, and structured set of data about a particular enterprise
 - Controlled by a database management system (DBMS)
- DBMS
 - Set of programs to access the data
 - An environment that is both *convenient* and *efficient* to use
- Database systems are used to manage collections of data that are:
 - Highly valuable
 - Relatively large
 - Accessed by multiple users and applications, often at the same time.
- A modern database system is a complex software system whose task is to manage a large, complex collection of data.
- Databases touch all aspects of our lives

Purpose of Database Management Systems

- DBMS contains information about a particular enterprise
 - Collection of interrelated data
 - Set of programs to access the data
 - An environment that is both *convenient* and *efficient* to use
- Database Applications:
 - Banking: transactions
 - Airlines: reservations, schedules
 - Universities: registration, grades
 - Sales: customers, products, purchases
 - Online retailers: order tracking, customized recommendations
 - Manufacturing: production, inventory, orders, supply chain
 - Human resources: employee records, salaries, tax deductions

1. Purpose of Database system:

What a DBMS Facilitates??

- *Define* a particular database in terms of its data types, structures, and constraints
- *Construct* or load the initial database contents on a secondary storage medium
- *Manipulating* the database:
 - Retrieval: Querying, generating reports
 - Modification: Insertions, deletions and updates to its content
 - Accessing the database through Web applications
- *Processing and sharing* by a set of concurrent users and application programs
 - yet, keeping all data valid and consistent

Purpose of Database Management Systems

- DBMS may additionally provide:
 - Protection or security measures to prevent unauthorized access
 - “Active” processing to take internal actions on data
 - Presentation and visualization of data
 - Maintenance of the database and associated programs over the lifetime of the database application

Example of a Database

- Mini-world for the example:
 - Part of a UNIVERSITY environment
- Some mini-world *entities*:
 - STUDENTs
 - COURSEs
 - SLOTSs (of COURSEs)
 - (Academic) DEPARTMENTs
 - INSTRUCTORs

Example of a Database

- Some mini-world *relationships*:
 - SLOTSs *are of specific* COURSEs
 - STUDENTs *take* COURSEs
 - COURSEs *have prerequisite* COURSEs
 - INSTRUCTORs *teach* SLOTSs
 - COURSEs *are offered by* DEPARTMENTs
 - STUDENTs *major in* DEPARTMENTs
- Note: The above entities and relationships are typically expressed in a conceptual data model, such as the entity-relationship (ER) data or UML class model

Example of a Simple Database

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

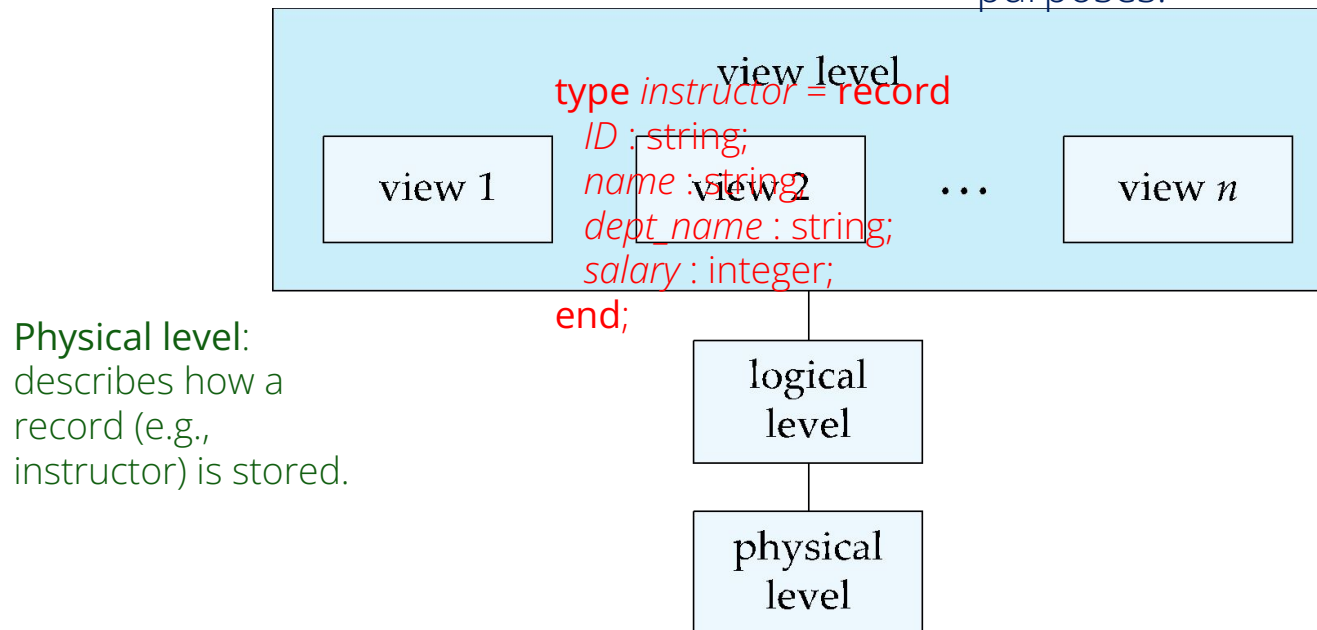
Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2

A database that stores student and course information.

View of Data in DBMS

View level: what application programs see; views can also hide information (such as an instructor's salary) for security purposes.



View of Data in DBMS

- **Physical level:** describes how a record (e.g., instructor) is stored.
- **Logical level:** describes data stored in database, and the relationships among the data.

```
type instructor = record
```

```
  ID : string;
```

```
  name : string;
```

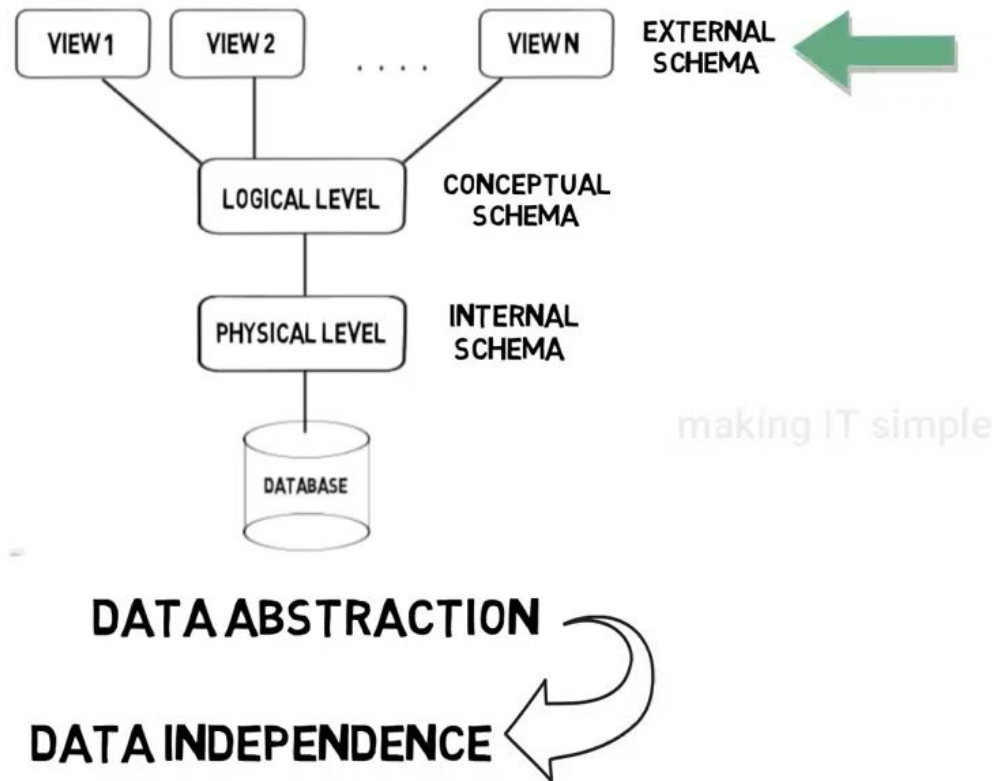
```
  dept_name : string;
```

```
  salary : integer;
```

```
end;
```

- **View level:** application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.

Data Independence Explained-Animation



View of Data in DBMS

Three-Schema Architecture

- Mappings among schema levels are needed to transform requests and data.
 - Programs refer to an external schema, and are mapped by the DBMS to the internal schema for execution.
 - Data extracted from the internal DBMS level is reformatted to match the user's external view (e.g. formatting the results of an SQL query for display in a Web page)

Data Independence

- **Logical Data Independence:**

- The capacity to change the conceptual schema without having to change the external schemas and their associated application programs.

- **Physical Data Independence:**

- The capacity to change the internal schema without having to change the conceptual schema.
- For example, the internal schema may be changed when certain file structures are reorganized or new indexes are created to improve database performance

Data Independence (continued)

- When a schema at a lower level is changed, only the **mappings** between this schema and higher-level schemas need to be changed in a DBMS that fully supports data independence.
- The higher-level schemas themselves are **unchanged**.
 - Hence, the application programs need not be changed since they refer to the external schemas.

DBMS Languages

- Data Definition Language (DDL)
- Data Manipulation Language (DML)
 - High-Level or Non-procedural Languages: These include the relational language SQL
 - May be used in a standalone way or may be embedded in a programming language
 - Low Level or Procedural Languages:
 - These must be embedded in a programming language

DBMS Languages- Data Definition Language(DDL)

- Data Definition Language (DDL):
 - Used by the DBA and database designers to specify the conceptual schema of a database.
 - Example:

```
create table instructor (  
    ID          char(5),  
    name        varchar(20),  
    dept_name   varchar(20),  
    salary      numeric(8,2))
```
 - In many DBMSs, the DDL is also used to define internal and external schemas (views).
 - In some DBMSs, separate **storage definition language (SDL)** and **view definition language (VDL)** are used to define internal and external schemas.
 - SDL is typically realized via DBMS commands provided to the DBA and database designers

DDL Commands in SQL

- CREATE to create a new table or database.
- ALTER for alteration.
- Truncate to delete data from the table.
- DROP to drop a table.
- RENAME to rename a table.

DBMS Languages- Data Manipulation Language(DML)

- Language for accessing and manipulating the data organized by the appropriate data model
 - DML also known as query language
 - DML commands (data sublanguage) can be *embedded* in a general-purpose programming language (host language), such as COBOL, C, C++, or Java.
 - A library of functions can also be provided to access the DBMS from a programming language
 - Alternatively, stand-alone DML commands can be applied directly (called a *query language*).

DML Commands

- SELECT – retrieve data from the a database
- INSERT – insert data into a table
- UPDATE – updates existing data within a table
- DELETE – deletes all records from a table, the space for the records remain
- MERGE – UPSERT operation (insert or update)
- CALL – call a PL/SQL or Java subprogram
- LOCK TABLE – control concurrency

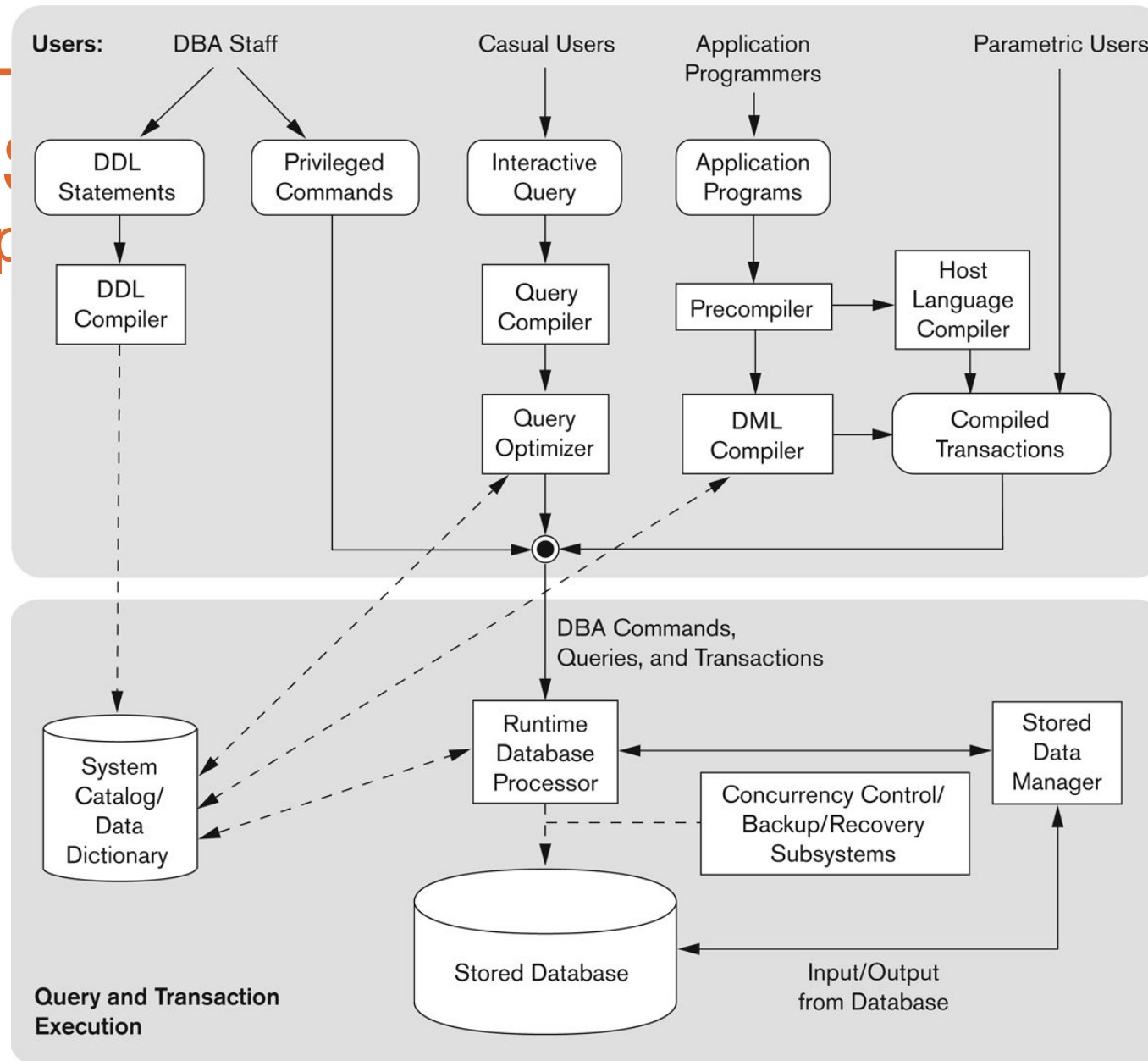
DBMS Languages-

Data Manipulation Language(DML)

- Programmer interfaces for embedding DML in a programming languages:
 - **Embedded Approach:** e.g embedded SQL (for C, C++, etc.), SQLJ (for Java)
 - **Procedure Call Approach:** e.g. JDBC for Java, ODBC for other programming languages
 - **Database Programming Language Approach:** e.g. ORACLE has PL/SQL, a programming language based on SQL; language incorporates SQL and its data types as integral components

Database S

(Levels, Mapping)



Database System Architecture

(Levels, Mappings, Database, users and DBA)

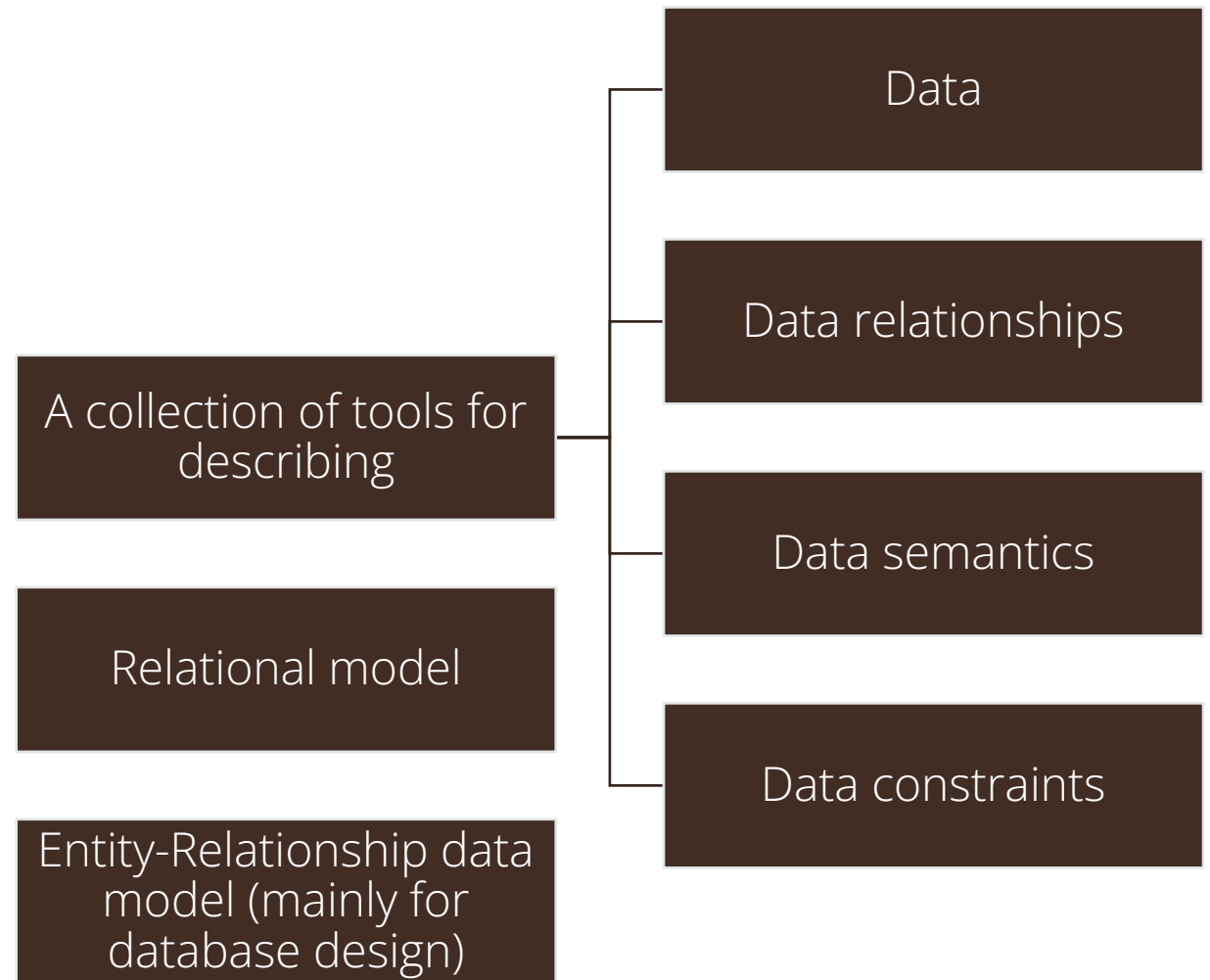
- **Users:**
 - **DBA staff** who works on defining the database and tuning it by making changes to its definition using the DDL and other privileged commands
 - **Casual users** who work with interactive interfaces to formulate queries,
 - **Application programmers** who create programs using some host programming languages, and
 - **Parametric users** who do data entry work by supplying parameters to predefined transactions.

Database System Architecture

(Levels, Mappings, Database, users and DBA)

- **The DDL compiler** processes schema definitions, specified in the DDL, and stores descriptions of the schemas (meta-data) in the DBMS catalog.
- **DBMS Catalog:** The catalog includes information such as the names and sizes of files, names and data types of data items, storage details of each file, mapping information among schemas, and constraints.
- **Stored data manager**, which in turn uses basic operating system services for carrying out low-level input/output (read/write) operations between the disk and main memory.
- **The runtime database processor** handles other aspects of data transfer, such as management of buffers in the main memory. Some DBMSs have their own buffer management module whereas others depend on the OS for buffer management.
- **Concurrency control** and **backup and recovery systems** are integrated into the working of the runtime database processor for purposes of transaction management.

Data Models



Data Models- Basic Building blocks-Schemas versus Instances

- Database Schema:
 - The *description* of a database.
 - Includes descriptions of the database structure, data types, and the constraints on the database.
- Schema Diagram:
 - An *illustrative* display of (most aspects of) a database schema.
- Schema Construct:
 - A *component* of the schema or an object within the schema, e.g., STUDENT, COURSE.

Data Models

- Data Model:

- A set of concepts to describe the *structure* of a database, the *operations* for manipulating these structures, and certain *constraints* that the database should obey.

- Data Model Structure and Constraints:

- Constructs are used to define the database structure
- Constructs typically include *elements* (and their *data types*) as well as groups of elements (e.g. *entity, record, table*), and *relationships* among such groups
- Constraints specify some restrictions on valid data; these constraints must be enforced at all times

Data Models- Basic Building blocks-Schemas versus Instances

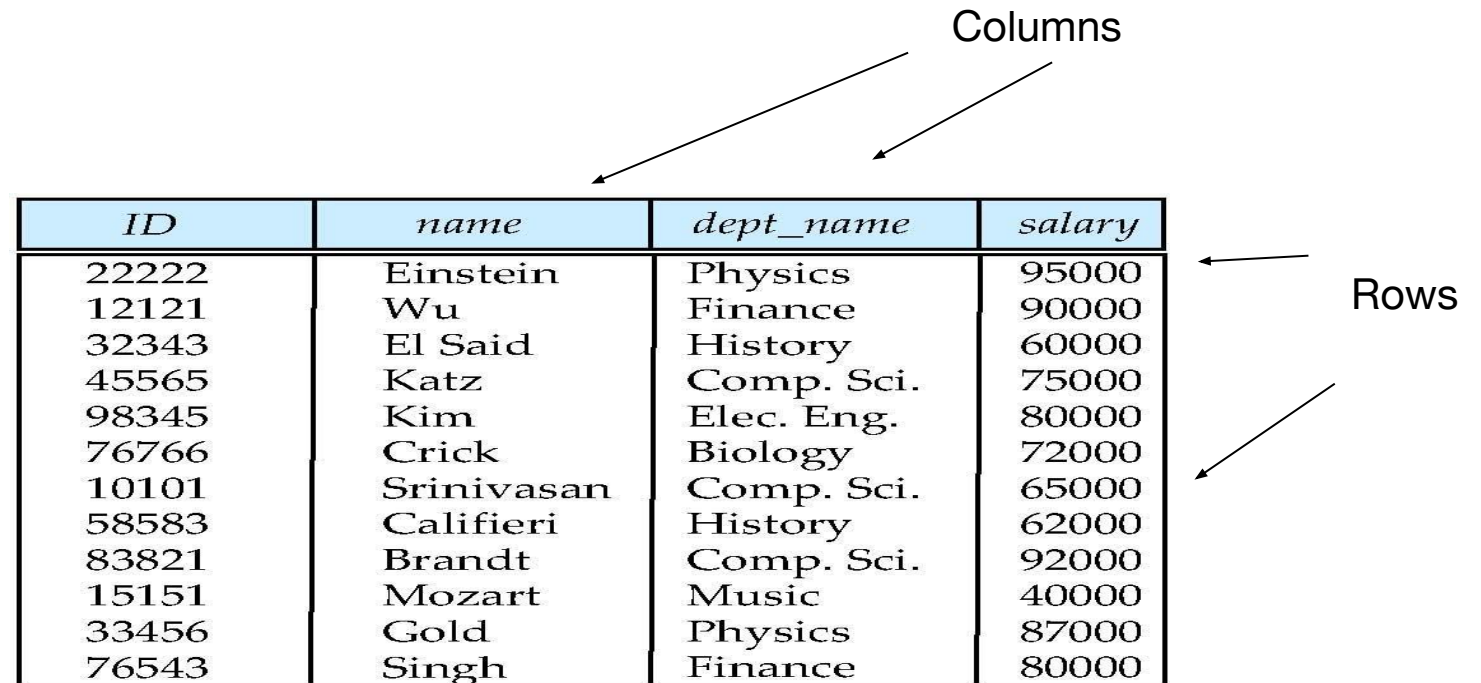
- Database State:
 - The actual data stored in a database at a *particular moment in time*. This includes the collection of all the data in the database.
 - Also called database instance (or occurrence or snapshot).
 - The term *instance* is also applied to individual database components, e.g. *record instance, table instance, entity instance*

Data Models- Basic Building blocks-Database Schema vs. Database State

- Database State:
 - Refers to the ***content*** of a database at a moment in time.
- Initial Database State:
 - Refers to the database state when it is initially loaded into the system.
- Valid State:
 - A state that satisfies the structure and constraints of the database.

Relational Model

- All the data is stored in various tables.
- Example of tabular data in the relational model



<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

A Sample Relational Database

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Data Models- Basic Building blocks-Database Schema vs. Database State (continued)

- Distinction
 - The *database schema* changes very infrequently.
 - The *database state* changes every time the database is updated.
- Schema is also called **intension**.
- State is also called **extension**.

Example of a Database Schema

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Figure 2.1

Schema diagram for the database in Figure 1.2.

Example of a database state

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

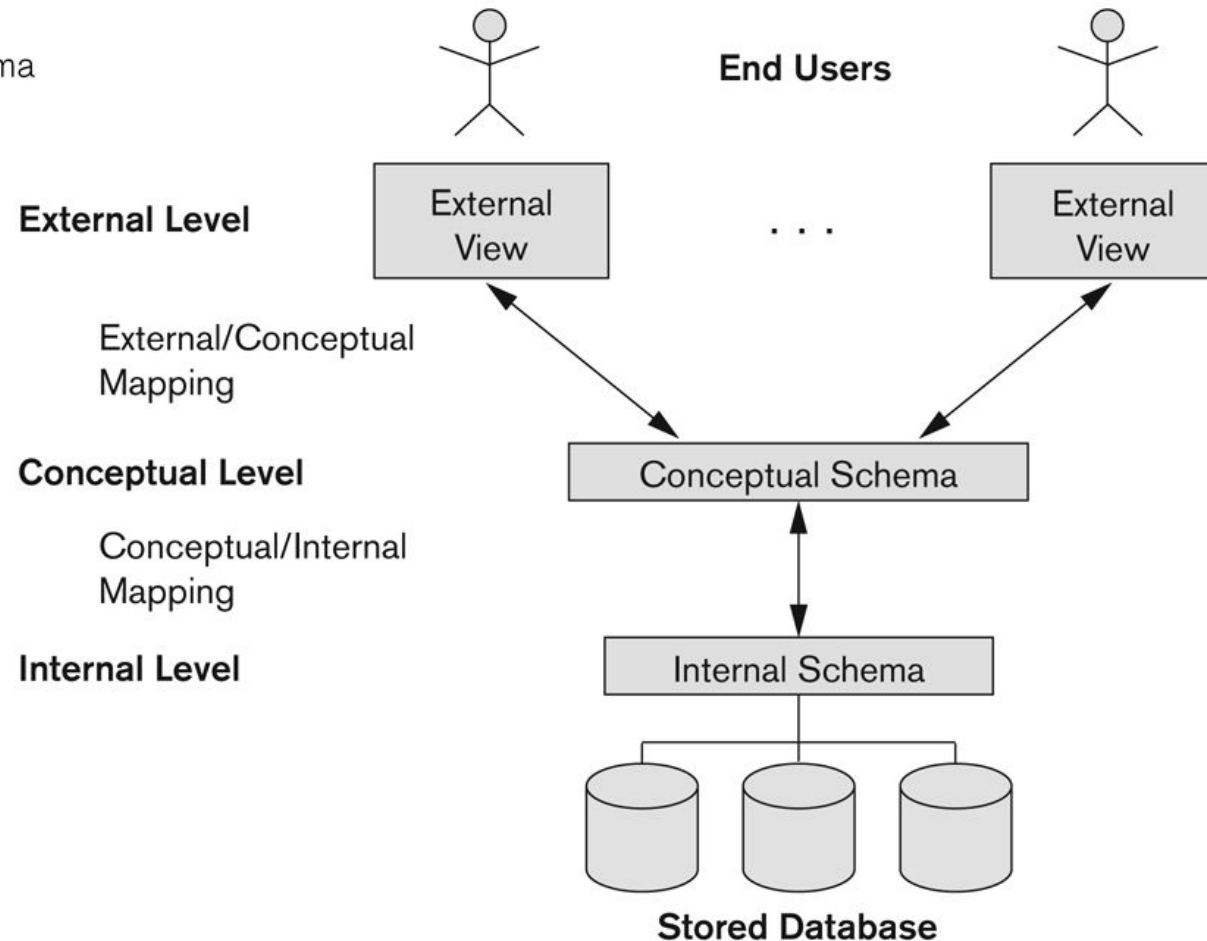
Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2
A database that stores
student and course
information.

Data models-Degrees of data abstraction-The three-schema architecture

Figure 2.2

The three-schema architecture.



Data models-Degrees of data abstraction-The three-schema architecture

- Mappings among schema levels are needed to transform requests and data.
 - Programs refer to an external schema and are mapped by the DBMS to the internal schema for execution.
 - Data extracted from the internal DBMS level is reformatted to match the user's external view (e.g. formatting the results of an SQL query for display in a Web page)

Data models-Degrees of data abstraction-The three-schema architecture

- When a schema at a lower level is changed, only the **mappings** between this schema and higher-level schemas need to be changed in a DBMS that fully supports data independence.
- The higher-level schemas themselves are **unchanged**.
 - Hence, the application programs need not be changed since they refer to the external schemas.

Database Design

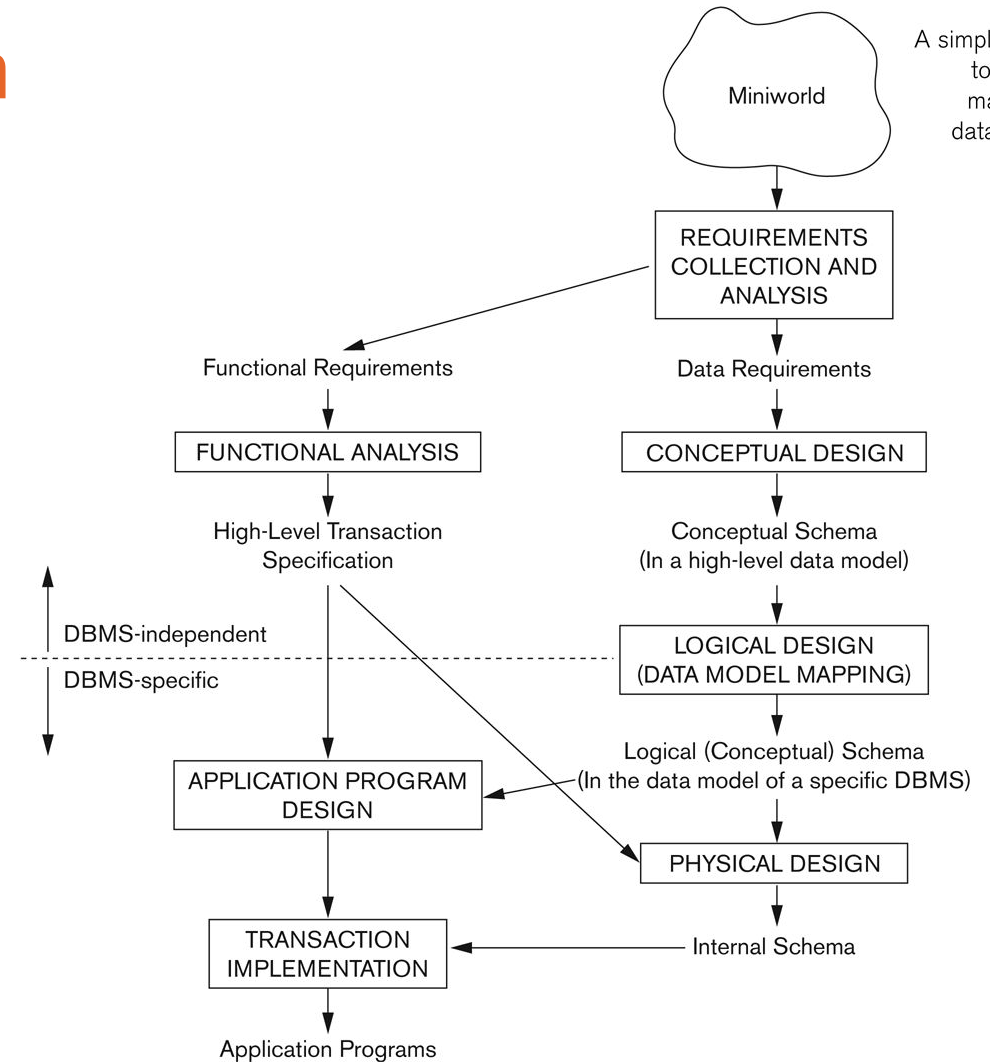
The process of designing the general structure of the database:

- Logical Design – Deciding on the database schema. Database design requires that we find a “good” collection of relation schemas.
 - Business decision – What attributes should we record in the database?
 - Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- Physical Design – Deciding on the physical layout of the database

Data Base design Approaches

- Need to produce a methodology to ensure that each of the relations in the database is “good”
- Two ways of doing so:
 - Entity Relationship Model (Chapter 7)
 - Models an enterprise as a collection of *entities* and *relationships*
 - Represented diagrammatically by an *entity-relationship diagram*:
 - Normalization Theory (Chapter 8)
 - Formalize what designs are bad, and test for them

Overview of Database Design



Database Design (Cont.)

- Is there any problem with this relation?

<i>ID</i>	<i>name</i>	<i>salary</i>	<i>dept_name</i>	<i>building</i>	<i>budget</i>
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Tutorial: COMPANY Database

- We need to create a database schema design based on the following (simplified) **requirements** of the COMPANY Database:
 - The company is organized into DEPARTMENTS. Each department has a name, number and an employee who *manages* the department. We keep track of the start date of the department manager. A department may have several locations.
 - Each department *controls* a number of PROJECTs. Each project has a unique name, unique number and is located at a single location.

Example COMPANY Database (Contd.)

- We store each EMPLOYEE's social security number, address, salary, sex, and birthdate.
 - Each employee *works for* one department but may *work on* several projects.
 - We keep track of the number of hours per week that an employee currently works on each project.
 - We also keep track of the *direct supervisor* of each employee.
- Each employee may *have* a number of DEPENDENTS.
 - For each dependent, we keep track of their name, sex, birthdate, and relationship to the employee.

ER Model or Entity Relationship Diagram

- **ER Model** stands for Entity Relationship Model is a high-level conceptual data model diagram.
- ER model helps to systematically analyze data requirements to produce a well-designed database.
- The ER Model represents real-world entities and the relationships between them.
- Creating an ER Model in DBMS is considered as a best practice before implementing your database.
- ER Modeling helps you to analyze data requirements systematically to produce a well-designed database.
- So, it is considered a best practice to complete ER modeling before implementing your database.

ER Model Concepts

- Entities and Attributes
 - Entities are specific objects or things in the mini-world that are represented in the database.
 - For example the EMPLOYEE John Smith, the Research DEPARTMENT, the ProductX PROJECT
 - Attributes are properties used to describe an entity.
 - For example an EMPLOYEE entity may have the attributes Name, SSN, Address, Gender, BirthDate
 - A specific entity will have a value for each of its attributes.
 - For example a specific employee entity may have Name='John Smith', SSN='123456789', Address='731, Fondren, Houston, TX', Gender='M', BirthDate='09-JAN-55'
 - Each attribute has a *value set* (or data type) associated with it – e.g. integer, string, subrange, enumerated type, ...

En'



ERD = Logical structure
of databases

Types of Attributes (1)

- Simple
 - Each entity has a single atomic value for the attribute. For example, status (T/F), Gender(M/F), work completion(Yes/No).
- Composite
 - The attribute may be composed of several components. For example:
 - Address(Apt#, House#, Street, City, State, PINCode, Country), or
 - Name(FirstName, MiddleName, LastName).
 - Composition may form a hierarchy where some components are themselves composite.
- Multi-valued
 - An entity may have multiple values for that attribute. For example, Color of a CAR or PreviousDegrees of a STUDENT.
 - Denoted as {Color} or {PreviousDegrees}.

Example of a composite attribute

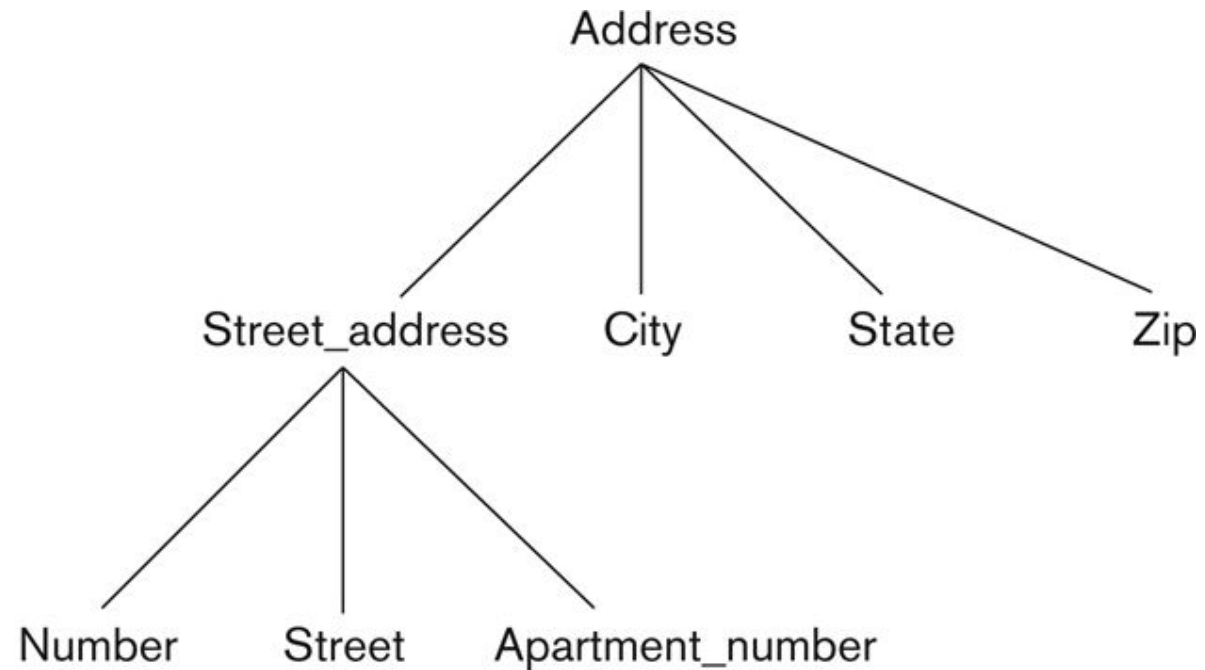
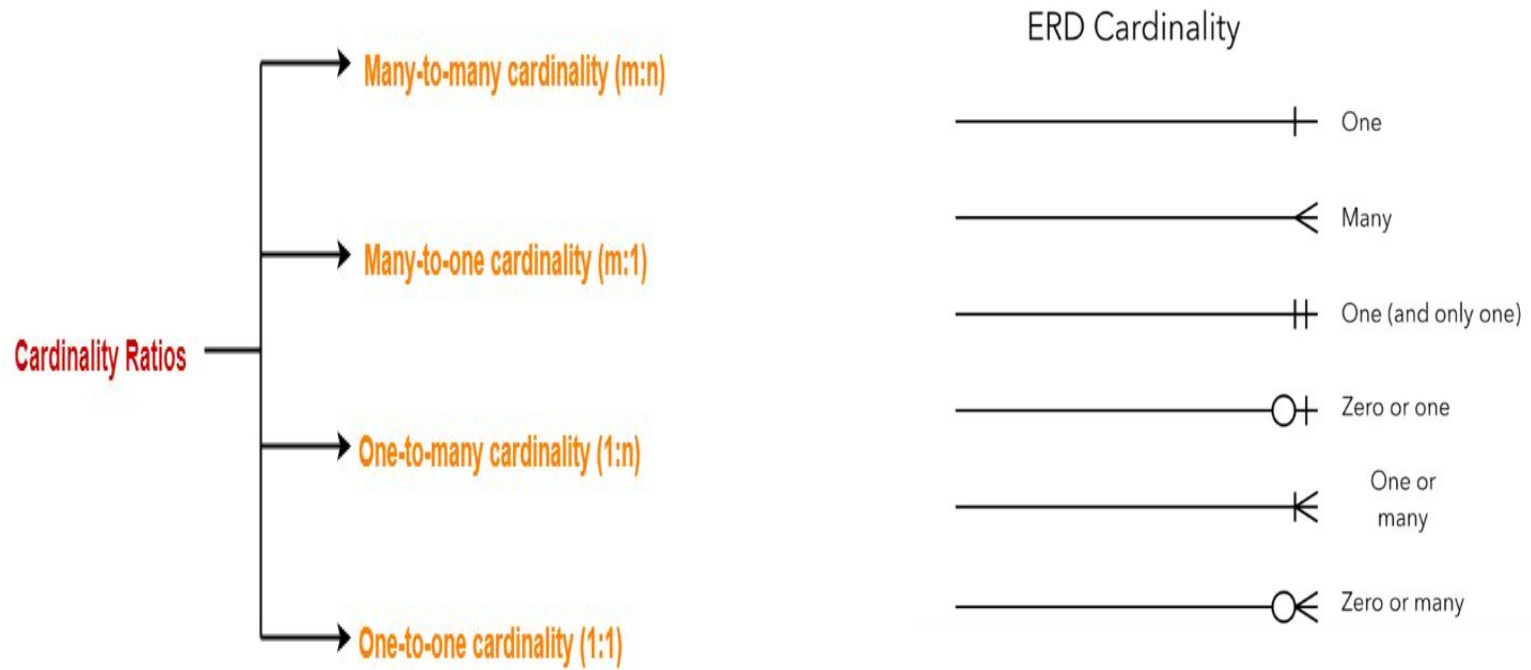


Figure 3.4
A hierarchy of
composite attributes.

Relationship of Attributes-Cardinality



Entity Types and Key Attributes (1)

- Entities with the same basic attributes are grouped or typed into an entity type.
 - For example, the entity type EMPLOYEE and PROJECT.
- An attribute of an entity type for which each entity must have a unique value is called a key attribute of the entity type.
 - For example, SSN of EMPLOYEE.

Entity Types and Key Attributes (2)

- A key attribute may be composite.
 - VehicleTagNumber is a key of the CAR entity type with components (Number, State).
- An entity type may have more than one key.
 - The CAR entity type may have two keys:
 - VehicleIdentificationNumber (popularly called VIN)
 - VehicleTagNumber (Number, State), aka license plate number.
- Each key is underlined

Displaying an Entity type

- ER Diagrams Symbols & Notations
- **Entity Relationship Diagram Symbols & Notations** mainly contains three basic symbols which are rectangle, oval and diamond to represent relationships between elements, entities and attributes. T
- here are some sub-elements which are based on main elements in ERD Diagram.
- ER Diagram is a visual representation of data that describes how data is related to each other using different ERD Symbols and Notations.

Displaying an Entity type

- Following are the main components and its symbols in ER Diagrams:
- **Rectangles:** This Entity Relationship Diagram symbol represents entity types
- **Ellipses :** Symbol represent attributes
- **Diamonds:** This symbol represents relationship types
- **Lines:** It links attributes to entity types and entity types with other relationship types
- **Primary key:** attributes are underlined
- **Double Ellipses:** Represent multi-valued attributes



Entity –Attribute-Relationship

-Real life examples



Entity Name

Entity

Person,place,object,event
or concept about which
data is to be maintained

Example: Car, Student



Attribute Name

Attribute

Property or characteristic of
an entity

Example: Color of car Entity
Name of Student Entity



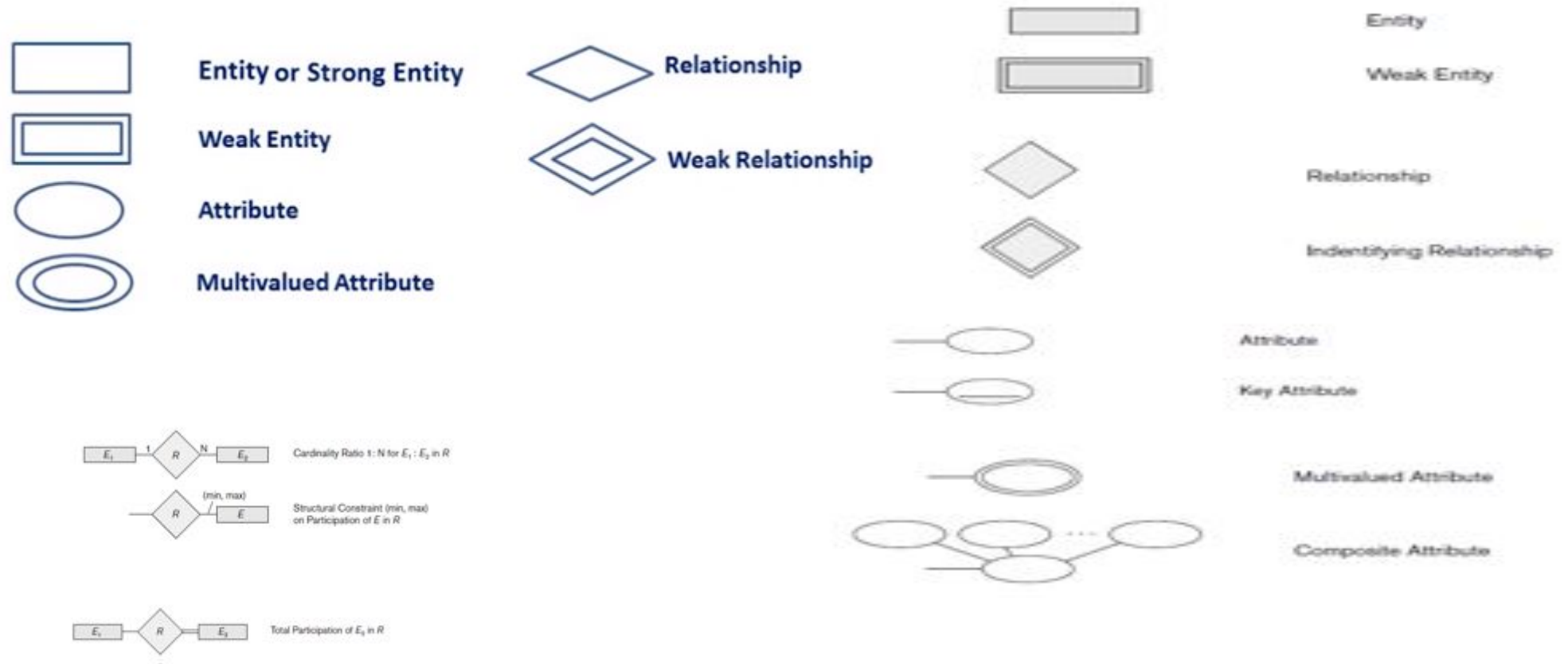
Relation



Association between the instances of one or
more entity types

Example: Blue Car Belongs to Student Jack

Displaying an Entity type



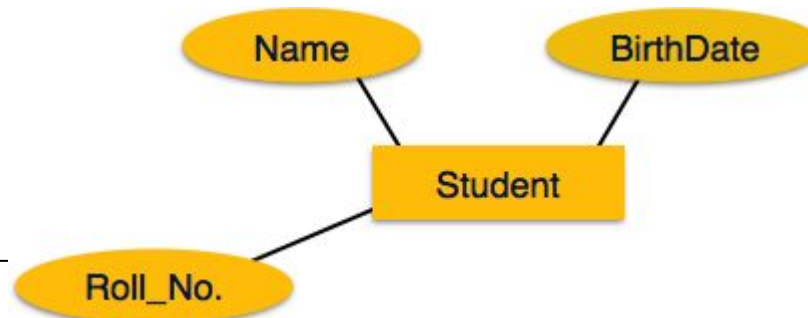
Entity, Attributes- Representation

Entities are represented by means of rectangles.
Rectangles are named with the entity set they represent.



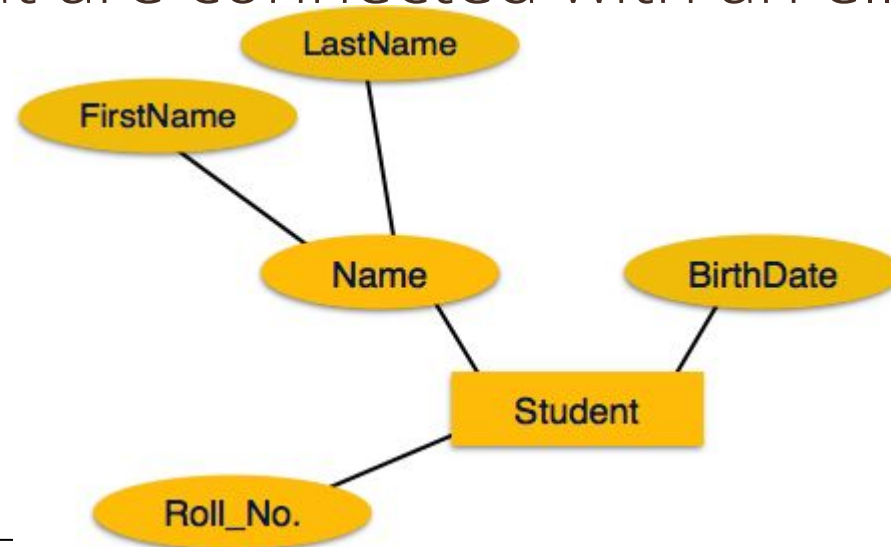
Attributes

Attributes are the properties of entities.
Attributes are represented by means of ellipses.
Every ellipse represents one attribute and is directly connected to its entity (rectangle).



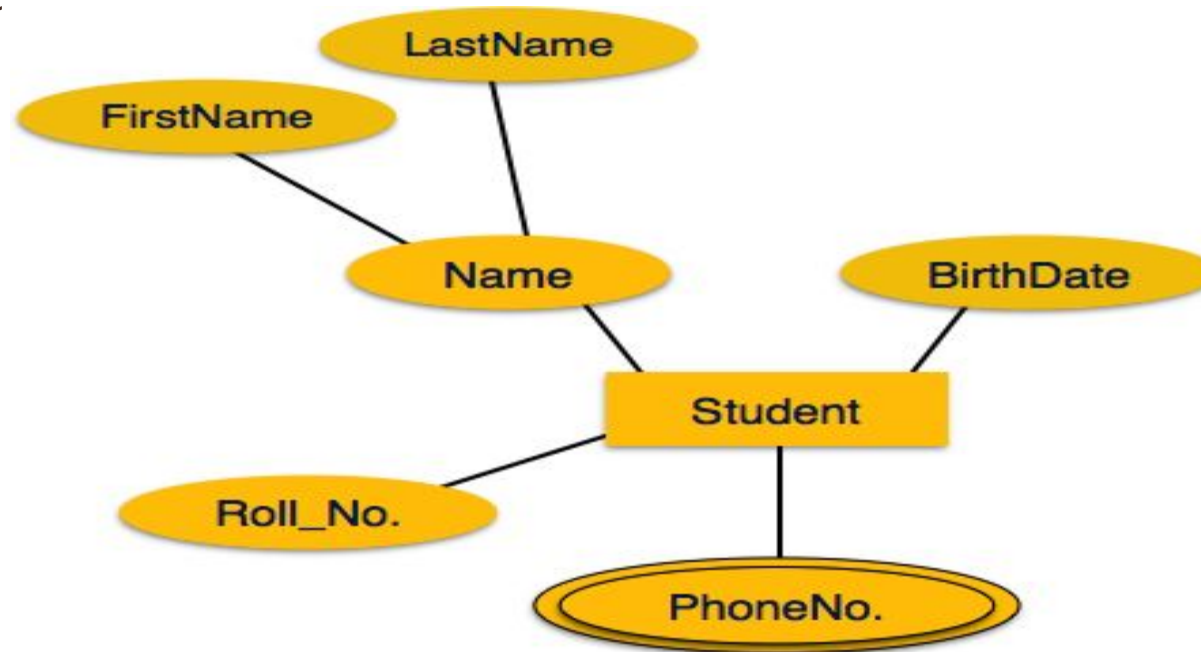
Composite Attributes- Representation

- If the attributes are **composite**, they are further divided in a tree like structure.
- Every node is then connected to its attribute.
- That is, composite attributes are represented by ellipses that are connected with an ellipse.



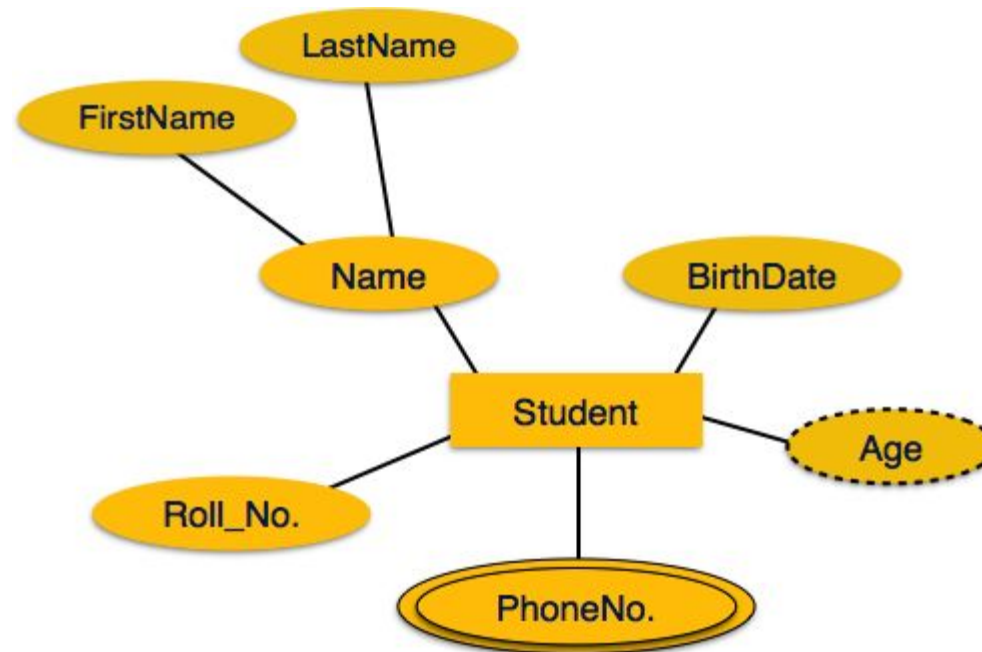
Multivalued Attributes- Representation

- Multivalued attributes are depicted by double ellipse



Derived Attributes- Representation

Derived attributes are depicted by dashed ellipse.



Relationship-Representation

- Relationships are represented by diamond-shaped box.
- Name of the relationship is written inside the diamond-box.
- All the entities (rectangles) participating in a relationship, are connected to it by a line.
- One-to-one

Relationship-Representation

One-to-One



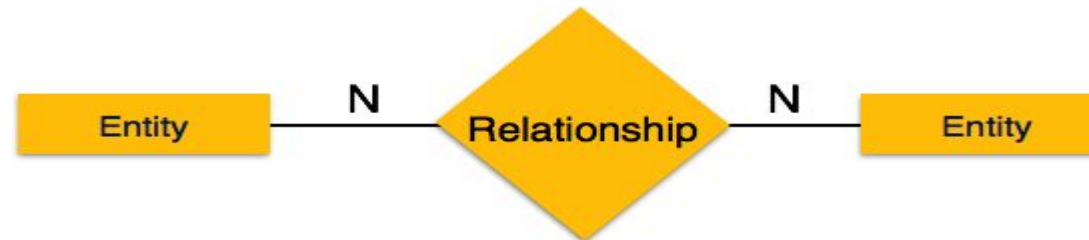
Many-to-One



One-to-Many



Many-to-Many

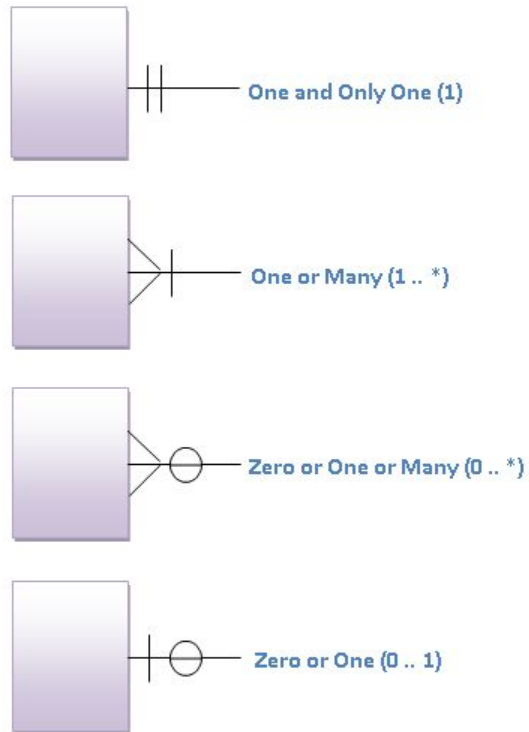


Cardinality-Data Modelling

A cardinality is the number of rows a table can have in the relationship.

Cardinality defines the possible number of occurrences in one entity which is associated with the number of occurrences in another

Crow Foot Notation Symbols

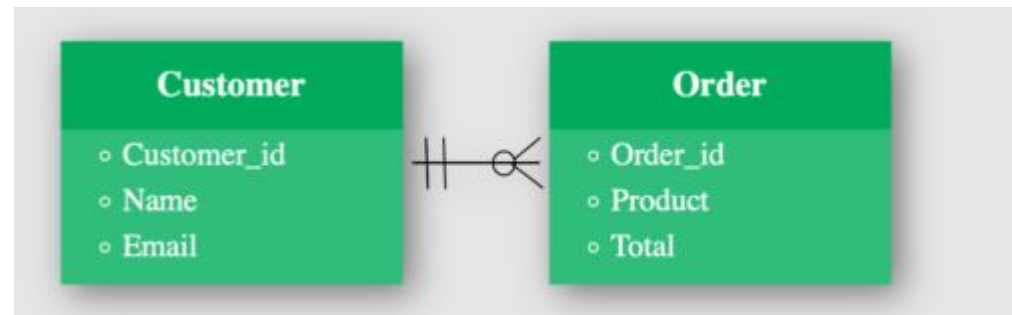
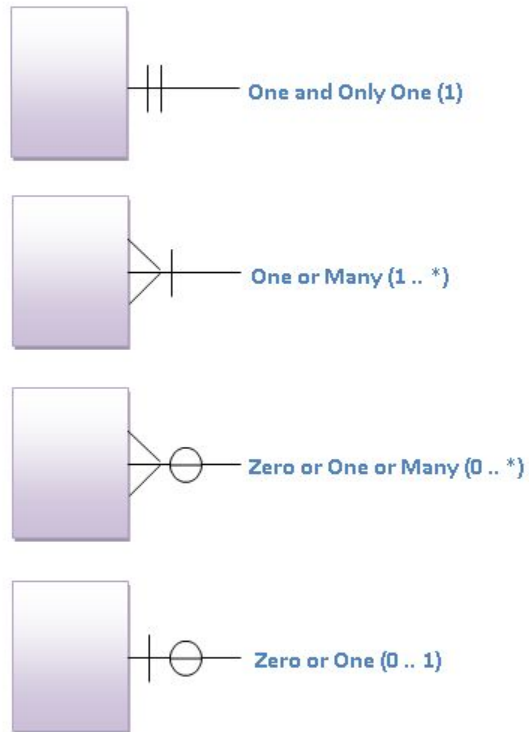


1. One and Only One
2. One or Many
3. Zero or One or Many
4. Zero or One

Cardinality-Data Modelling

A cardinality is the number of rows a table can have in the relationship.

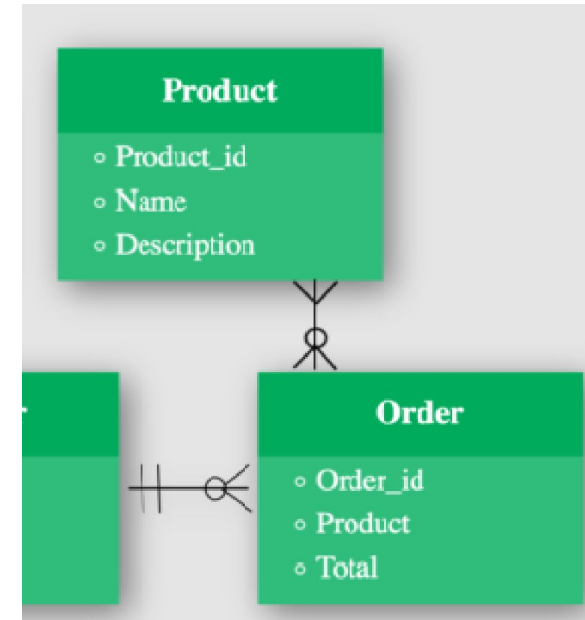
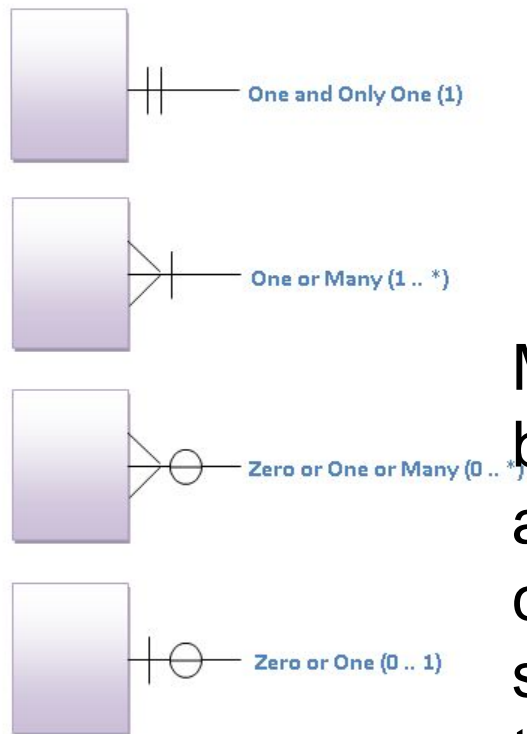
Crow Foot Notation Symbols



The relationship line shows that a customer can have zero or many orders, while an order can be associated with one and only one customer. The verb used for the relationship can be “places”, as in the customer places an order.

Cardinality-Data Modelling

Crow Foot Notation Symbols

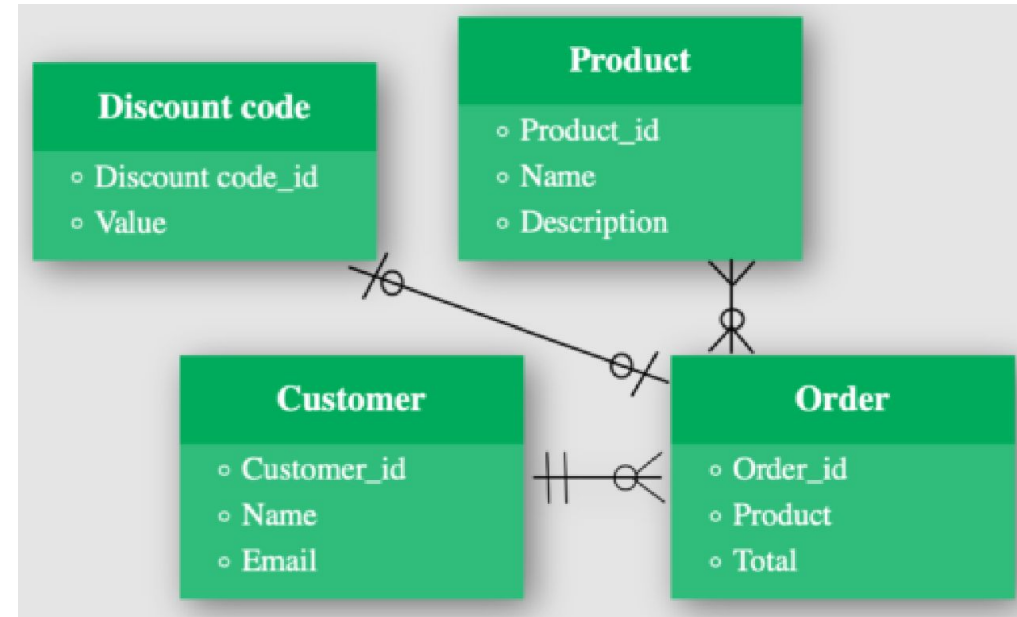
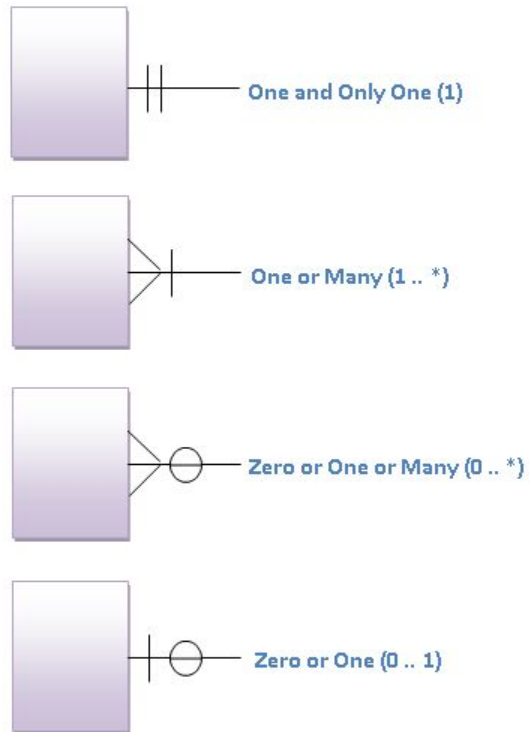


Moving on to product and order, a product can be associated with zero or many orders, while an order must have one or many products (you can't have an order without ordering something!). For product and order, we can use the verb "is added to" to describe how a product is added to an order

Cardinality-Data Modelling

A cardinality is the number of rows a table can have in the relationship.

Crow Foot Notation Symbols



Orders and discount codes have a zero or one relationship: a customer might not input a discount code, but if they do, they can only use one at a time. Going back down the line the other way, an order might not have any discount code associated with it, but if it does, it can only have one. A discount code is “applied to” an order.

ERD-Car

(a)

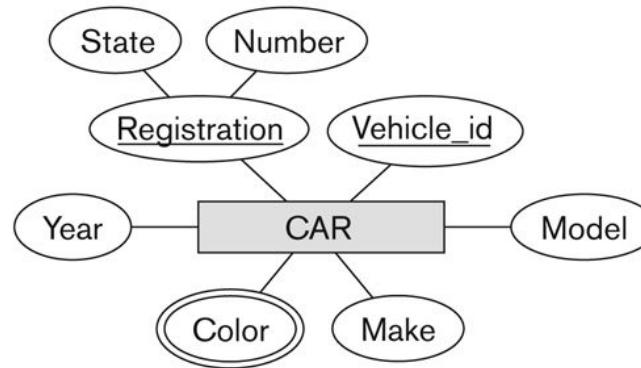


Figure 3.7

The CAR entity type with two key attributes, Registration and Vehicle_id. (a) ER diagram notation. (b) Entity set with three entities.

(b)

CAR
Registration (Number, State), Vehicle_id, Make, Model, Year, {Color}

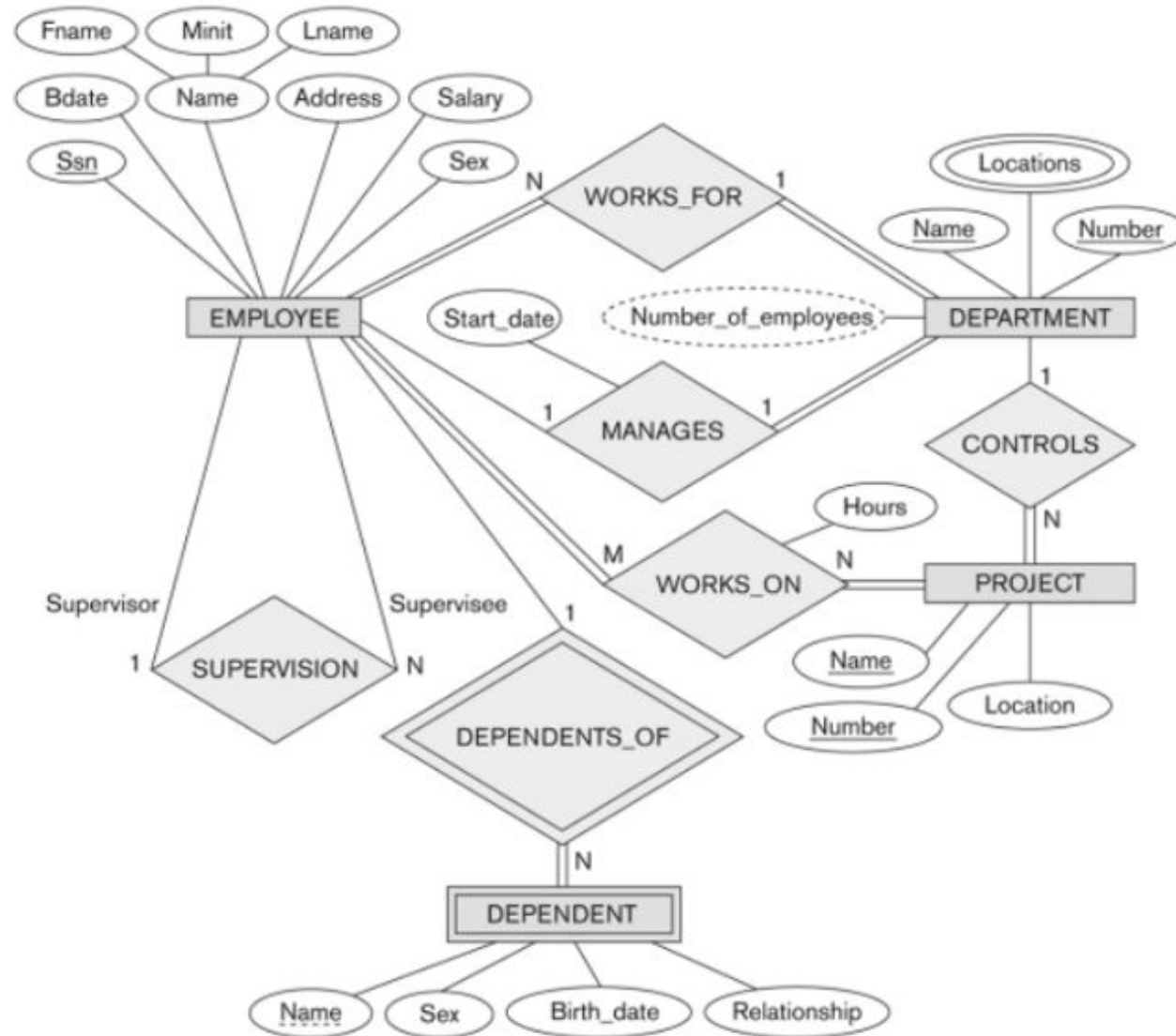
CAR₁
((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 2004 {red, black})

CAR₂
((ABC 123, NEW YORK), WP9872, Nissan Maxima, 4-door, 2005, {blue})

CAR₃
((VSY 720, TEXAS), TD729, Chrysler LeBaron, 4-door, 2002, {white, blue})

⋮

ERD-Relationships-Employee Mini World



Initial Design of Entity Types:

EMPLOYEE, DEPARTMENT, PROJECT, DEPENDENT

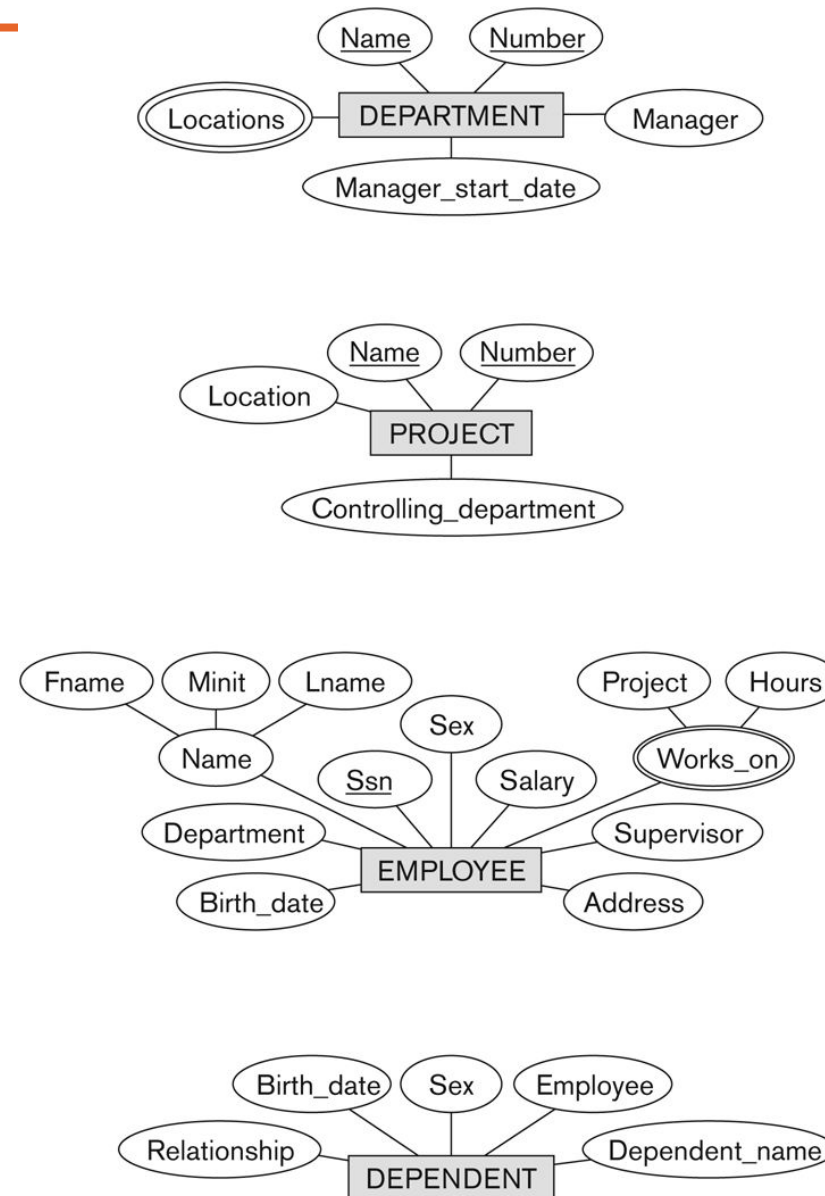


Figure 3.8
Preliminary design of entity types for the COMPANY database. Some of the shown attributes will be refined into relationships.

Entity Set

- Each entity type will have a collection of entities stored in the database
 - Called the **entity set**
- Previous slide shows three CAR entity instances in the entity set for CAR
- Same name (CAR) used to refer to both the entity type and the entity set
- Entity set is the current *state* of the entities of that type that are stored in the database

Initial Design of Entity Types for the COMPANY Database Schema

- Based on the requirements, we can identify four initial entity types in the COMPANY database:
 - DEPARTMENT
 - PROJECT
 - EMPLOYEE
 - DEPENDENT
- Their initial design is shown on the following slide
- The initial attributes shown are derived from the requirements description

Initial Design of Entity Types:

EMPLOYEE, DEPARTMENT, PROJECT, DEPENDENT

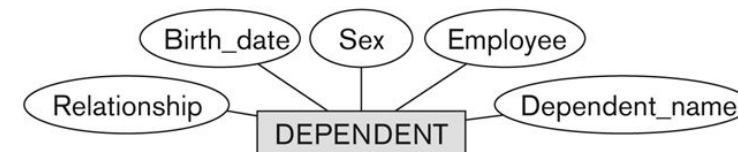
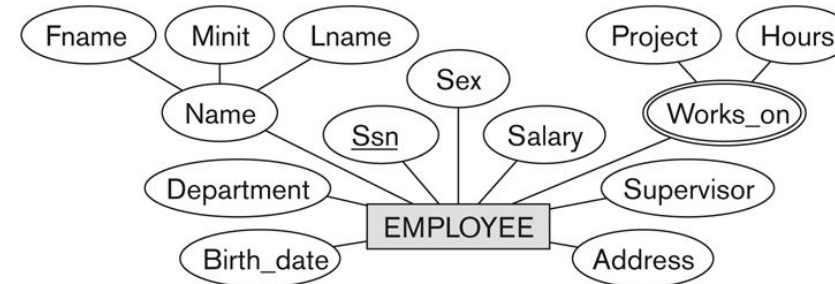
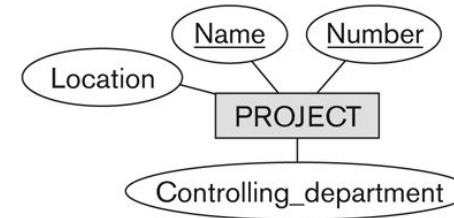
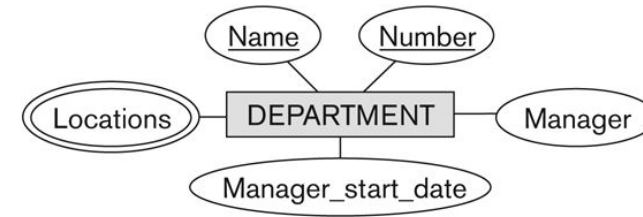


Figure 3.8
Preliminary design of entity types for the COMPANY database. Some of the shown attributes will be refined into relationships.

Refining the initial design by introducing relationships

- The initial design is typically not complete
- Some aspects in the requirements will be represented as **relationships**
- ER model has three main concepts:
 - Entities (and their entity types and entity sets)
 - Attributes (simple, composite, multivalued)
 - Relationships (and their relationship types and relationship sets)
- We introduce relationship concepts next

Relationships and Relationship Types (1)

- A **relationship** relates two or more distinct entities with a specific meaning.
 - For example, EMPLOYEE John Smith *works on* the ProductX PROJECT, or EMPLOYEE Franklin Wong *manages* the Research DEPARTMENT.
- Relationships of the same type are grouped or typed into a **relationship type**.
 - For example, the WORKS_ON relationship type in which EMPLOYEEs and PROJECTs participate, or the MANAGES relationship type in which EMPLOYEEs and DEPARTMENTs participate.
- The degree of a relationship type is the number of participating entity types.
 - Both MANAGES and WORKS_ON are *binary* relationships.

Relationship type vs. relationship set (1)

- Relationship Type:
 - Is the schema description of a relationship
 - Identifies the relationship name and the participating entity types
 - Also identifies certain relationship constraints
- Relationship Set:
 - The current set of relationship instances represented in the database
 - The current *state* of a relationship type

Relationship type vs. relationship set (2)

- Previous figures displayed the relationship sets
- Each instance in the set relates individual participating entities – one from each participating entity type
- In ER diagrams, we represent the *relationship type* as follows:
 - Diamond-shaped box is used to display a relationship type
 - Connected to the participating entity types via straight lines

Refining the COMPANY database schema by introducing relationships

- By examining the requirements, six relationship types are identified
- All are *binary* relationships(degree 2)
- Listed below with their participating entity types:
 - WORKS_FOR (between EMPLOYEE, DEPARTMENT)
 - MANAGES (also between EMPLOYEE, DEPARTMENT)
 - CONTROLS (between DEPARTMENT, PROJECT)
 - WORKS_ON (between EMPLOYEE, PROJECT)
 - SUPERVISION (between EMPLOYEE (as subordinate), EMPLOYEE (as supervisor))
 - DEPENDENTS_OF (between EMPLOYEE, DEPENDENT)

ER DIAGRAM – Relationship Types are:

WORKS_FOR, MANAGES, WORKS_ON, CONTROLS, SUPERVISION, DEPENDENTS_OF

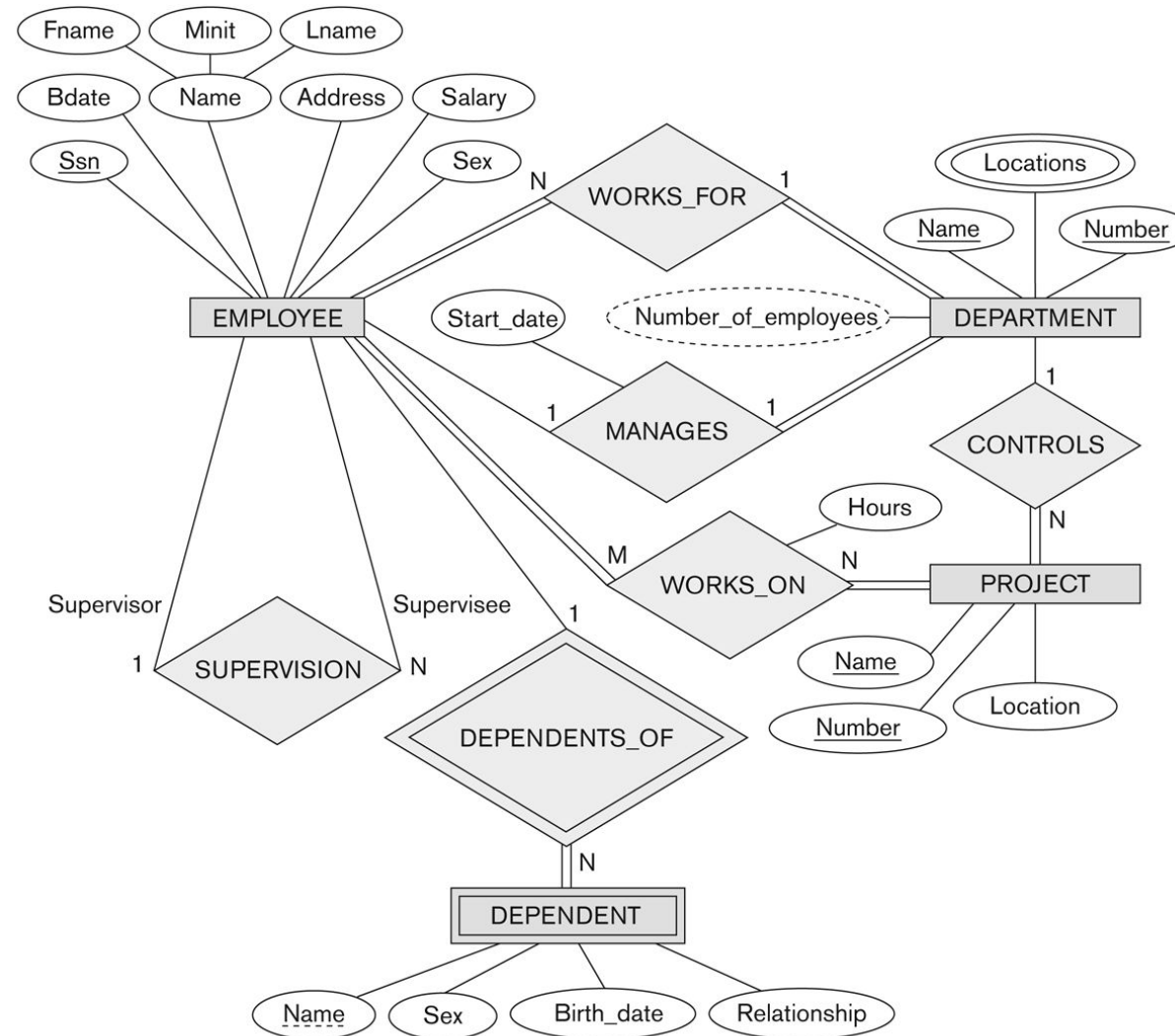


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Discussion on Relationship Types

- In the refined design, some attributes from the initial entity types are refined into relationships:
 - Manager of DEPARTMENT -> MANAGES
 - Works_on of EMPLOYEE -> WORKS_ON
 - Department of EMPLOYEE -> WORKS_FOR
 - etc
- In general, more than one relationship type can exist between the same participating entity types
 - MANAGES and WORKS_FOR are distinct relationship types between EMPLOYEE and DEPARTMENT
 - Different meanings and different relationship instances.

Recursive Relationship Type

- An relationship type whose with the same participating entity type in **distinct roles**
- Example: the SUPERVISION relationship
- EMPLOYEE participates twice in two distinct roles:
 - supervisor (or boss) role
 - supervisee (or subordinate) role
- Each relationship instance relates two distinct EMPLOYEE entities:
 - One employee in *supervisor* role
 - One employee in *supervisee* role

Weak Entity Types

- An entity that does not have a key attribute
- A weak entity must participate in an identifying relationship type with an owner or identifying entity type
- Entities are identified by the combination of:
 - A partial key of the weak entity type
 - The particular entity they are related to in the identifying entity type
- **Example:**
 - A DEPENDENT entity is identified by the dependent's first name, *and* the specific EMPLOYEE with whom the dependent is related
 - Name of DEPENDENT is the *partial key*
 - DEPENDENT is a *weak entity type*
 - EMPLOYEE is its identifying entity type via the identifying relationship type DEPENDENT_OF